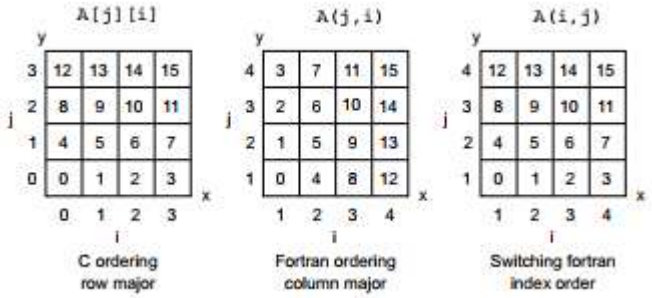
Chapter 4

- Design data to perform well
  - Layout internal matters for Cache / network etc.
  - Data movement becoming more important as parallelization is required
- Code type matters for performance
  - OOP has long call stack
  - Functional uses sequence of operations on single call stack
- Check CppCon presentation "Data-oriented design and C++"
  - https://www.youtube.com/watch?v=rX0ItVEVjHc
  - Data-oriented design trys to optimize the data for performance by controlling layout in memory for CPU / Cache
  - Natural in Fortran
- Data-oriented looks like…
  - Operate on array, not individual elements
  - Arrays better than structures
  - Inline as much as possible
  - Direct control of memory
  - Contiguous arrays rather than linked-list, keep all memory in one place
- Multi-dimensional arrays can be indexed multiple ways in memory
  - 

  - Slicing this array can be done by copying or making the dope (info) vector
- Continuous arrays in C take a special way to allocate multi-dimensional arrays that is not commonly taught
  - Page 92
- Array of Structures (AoS) vs Structure of Arrays (SoA)
  - Depending on access pattern it can be better to either have elements in an array or have all similar elements in a continuous array, grouped as a structure
  - RGBRGBRGB vs RRRGGGBBB in memory
- Array of Structures of Arrays (AoSoA)
  - Mixed method which allows for tiling of SoA
- Cache misses hurt performance greatly.
  - Compulsory: Data needed to be brought in when first used
  - Capacity: Cache runs out of space
  - Conflict: Data loaded to same location, need to load cache line twice since both datums are needed at same time
- Case Study
  - Need to collect data on
    - Memops
    - Flops
    - Branching
    - Loops
    - Contiguous vs non-contiguous loading of memory/array

- Simulate material in cells as example
- Full matrix cell-centric: Assume all cells can contain all materials, track dense data structure of whats in cell
- Full matrix material-centric: Assume all materials can be in all cells, track dense data structure of where materials are
- Cell-centric compressed sparse: Track cells as number of materials, followed by materials, followed by next cell
  - Extra memory can be allocated to enable easy addition on new materials
- Material centric compressed sparse: Track materials as how many locations they are in, followed by what cells they are in, followed by next material
  - Similarly, extra memory can be allocated to make it easy to add material to new cell

Table 4.1 The sparse data structures are faster and use less memory than the full 2D matrices.

	Memory load (MBs)	Flops	Estimated run time	Measured run time
Cell-centric full	424	3.1	67.2	108
Material-centric full	1632	101	122	164
Cell-centric compressed sparse	6.74	.24	.87	1.4
Material-centric compressed sparse	74	3.1	5.5	9.6

- Execution Cache Memory (ECM)

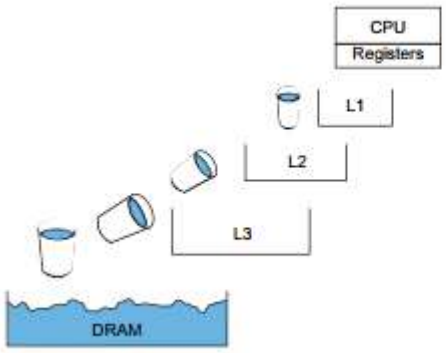


Figure 4.21 The movement of data between cache levels is a series of discrete operations, more like a bucket brigade than a flow through a pipe. The details of the hardware and how many loads can be issued at each level and in each direction largely impact the efficiency of loading data through the cache hierarchy.

  - Understanding flow of data through memory to cache to CPU registers is critical to understand maximum performance and bandwidth limits
- Network
  - Data transfer between nodes can be a limiting factory as well.
  - HPC Benchmark for latency and bandwidth at http://icl.cs.utk.edu/hpcc/hpcc_results_lat_band.cgi
  - It is possible to optimize the network message methods to maximize bandwidth beyond simply sending data